

# Deployment Guide

## Wheat Water Productivity Dashboard

Ethiopian Institute of Agricultural Research · IWMI East Africa · FAO WaPOR Phase II  
Generated 2026-07-29

## Deployment — Railway

Staging first, then production. Both environments run the same image from the same `Dockerfile`; only environment variables differ.

### What the image does

`Dockerfile` is a two-stage build:

1. `node:20-slim` installs the frontend dependencies and runs `vite build`.
2. `python:3.11-slim` installs the backend requirements, copies the built bundle to `frontend/dist`, and **trains the model during the build** so the image ships ready to serve.

Training at build time rather than at startup is deliberate: a cold container would otherwise spend ~40 s training before its first response, which is long enough to fail the platform health check and put the service into a restart loop. The container also installs `libgomp1` — LightGBM's OpenMP runtime, which the slim Python image omits and without which `import lightgbm` fails.

The service runs as an unprivileged user and binds `$PORT`, which Railway injects.

### Environment variables

| VARIABLE                      | REQUIRED                        | PURPOSE                                                                                                                                                                               |
|-------------------------------|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>PORT</code>             | injected by Railway             | Listen port                                                                                                                                                                           |
| <code>WWP_ADMIN_TOKEN</code>  | for model management            | Enables <code>POST /api/model/retrain</code> . <b>Without it that route returns 503</b> — the service fails closed rather than exposing model replacement.                            |
| <code>WWP_MODELS_DIR</code>   | if retraining in place          | Absolute path to a mounted volume for model artifacts. Without it artifacts live on the container filesystem and any retrained model is <b>lost on the next restart or redeploy</b> . |
| <code>WWP_CORS_ORIGINS</code> | only for cross-origin embedding | Comma-separated allowlist. The dashboard is served from the same origin, so this is unnecessary unless the EIAR site calls the API from another host.                                 |

### First deploy — staging

The Railway CLI needs an interactive browser login for `railway login`; the rest runs unattended.  
**Commands below are PowerShell** (the project's development environment); bash equivalents follow in

the next section.

```
npm install -g @railway/cli
railway login                # opens a browser

railway init                 # creates the project
railway environment new staging
railway environment staging
railway add --service wwp-dashboard

railway up --service wwp-dashboard --environment staging
railway domain               # mint a public URL
```

Model management stays disabled until you set a token. Generate and set one only when EIAR needs retraining enabled — this prints nothing to the terminal:

```
$tok = -join ((1..32) | ForEach-Object { '{0:x}' -f (Get-Random -Max 16) })
railway variables --set "WWP_ADMIN_TOKEN=$tok" --service wwp-dashboard
$env:WWP_ADMIN_TOKEN = $tok          # so the test suites can use it
```

## bash / zsh equivalents

```
railway variables --set "WWP_ADMIN_TOKEN=$(openssl rand -hex 24)" --service wwp-dashboard
export WWP_ADMIN_TOKEN=...          # PowerShell uses $env:NAME = "..." instead
```

Setting an environment variable is the one place the two shells differ enough to trip you up:

|                | POWERSHELL                                   | BASH / ZSH                                |
|----------------|----------------------------------------------|-------------------------------------------|
| Set a variable | <code>\$env:WWP_BASE = "https://host"</code> | <code>export WWP_BASE=https://host</code> |
| Read it back   | <code>\$env:WWP_BASE</code>                  | <code>\$WWP_BASE</code>                   |

PowerShell has no `export`, and **no spaces around the `=`**.

Railway reads `railway.json`, which pins the Dockerfile builder, sets the health check to `/api/health` with a 300 s timeout (the image build trains the model, so boot itself is fast, but the generous timeout covers a slow cold start), and **fixes the service at one replica**.

One replica is a correctness requirement, not a cost choice. The upload registry and the completed-run store are in-process dictionaries, so with two replicas a boundary uploaded through one instance is invisible to the other and CSV export links break at random. See the ToR Coverage document for the PostGIS work that lifts this constraint.

## Verify staging

Point the test suites at the deployed URL — they need no local server:

```
export WWP_BASE=https://<your-staging-domain>
python tests/test_api_e2e.py          # 46 checks, or 49 with WWP_ADMIN_TOKEN set

cd tests && npm install playwright && npx playwright install chromium
node test_ui.mjs                      # 51 browser checks against the live site
```

The API suite covers all three AOI journeys, caching, idempotency and every validation path, so a green run against staging is a real smoke test rather than a liveness ping.

## Promote to production

```
railway environment production
railway variables --set "WWP_ADMIN_TOKEN=<a different secret>"
railway up
railway domain
```

Use a **different** admin token per environment so a staging leak cannot touch the production model.

## Retraining in a deployment

Model artifacts live on the container filesystem by default, which Railway replaces on every deploy. To retrain against real field data and keep the result:

1. Attach a Railway volume, mounted at (say) `/data/models`.
2. Set `WWP_MODELS_DIR=/data/models`.
3. Restart. The service finds the volume empty and trains a fresh `v0001` into it, after which every promoted version persists across deploys.

Then retrain with:

```
curl -X POST https://<domain>/api/model/retrain \
  -H "X-Admin-Token: $WWP_ADMIN_TOKEN" \
  -F file=@field_observations.csv
```

The response reports the candidate's holdout metrics beside the active model's on the same split, and whether it was promoted. A candidate that would degrade RMSE is rejected and the running model keeps serving.

## Before this is a public EIAR service

Beyond the environment setup above, the gaps listed in the ToR Coverage document still apply — chiefly the live WaPOR provider (every value served today is synthetic), PostGIS for the upload and run stores, and Redis for the cache if the replica count ever rises above one.